

OPD-Evolver: Cultivating Holistic Agent Evolver via On-Policy Distillation

Guibin Zhang[†] Xun Xu[†] Yanwei Yue Zikun Su Wangchunshu Zhou

Xiaobin Hu[‡] Shuicheng Yan[‡]

[‡] Corresponding Authors [†] Equal Contribution

 LV-NUS Lab  FDU  PKU  Bytedance Inc.

 GitHub  Hugging Face

Abstract

Memory has become a standard substrate for self-evolving agents, yet retaining experience is not the same as learning how to evolve through it. Existing memory agents can store trajectories, retrieve reflections, or accumulate skills, but often lack the holistic competence to select useful experience, act on it, write reusable knowledge, and maintain a growing repository. We introduce OPD-Evolver, a slow-fast co-evolution framework that cultivates such an agent evolver through on-policy self-distillation. In the fast loop, OPD-Evolver interacts with a four-level memory hierarchy to read, use, write, and maintain experience for rapid test-time evolution. In the slow loop, outcome-calibrated memory attribution and privileged hindsight distill these four abilities into the deployable policy. Across multi-domain benchmarks, OPD-Evolver surpasses memory systems such as ReasoningBank by up to 11.5%, and training-based methods such as Skill0 by $\sim 5.8\%$. Further analysis shows that OPD-Evolver internalizes high-value experience and memory management, enabling OPD-Evolver-9B to challenge giant counterparts such as QWEN3.5-397B-A17B and STEP-3.5-FLASH, pointing beyond memory-augmented agents toward genuinely qualified agent evolvers.

1 Introduction

What defines a self-evolving agent? A natural answer is memory, since preserving past trajectories, retrieving prior lessons, and reusing accumulated skills seem to provide the material basis for improvement over time (Hu et al., 2025; Fang et al., 2025a). This intuition has made memory an indispensable component of modern agentic foundation models (Team et al., 2025; MiniMax, 2026) and agent systems (Pan et al., 2026; Zhou et al., 2026), especially in interactive environments where failures reveal latent constraints, successful rollouts

expose reusable strategies, and rewards ground behavior in environmental feedback (Wu et al., 2024; Zheng et al., 2025b; He et al., 2026). Yet memory is only the substrate of self-evolution, not its definition. Despite the proliferation of memory systems (Zhang et al., 2025a; Ouyang et al., 2025), skill libraries (Zheng et al., 2025a; Lu et al., 2026; Xia et al., 2026), reflection mechanisms (Liang et al., 2024), and self-improvement pipelines (Wu et al., 2025b; Wei et al., 2025), the notion of “self-evolution” remains under-specified: many agents can retain experience or expose it to the prompt, while far fewer can convert it into sustained behavioral improvement. In this work, we define an *agent evolver* as an agent that systematically transforms interaction history and feedback into persistent improvements in future behavior.

Existing work has approached this objective from diverse perspectives. Memory-augmented agents store trajectories, reflections, tips, or lessons and inject them into later prompts (Shinn et al., 2023; Zhao et al., 2024; Chhikara et al., 2025). Skill-augmented agents, which we regard as a structured instantiation of memory, distill experience into reusable strategies, tools, or procedures (Xia et al., 2026; Zhang et al., 2026a; Lu et al., 2026; Shi et al., 2026). Other methods more directly parameterize experience by training on collected trajectories through supervised fine-tuning (SFT), reinforcement learning (RL), or on-policy distillation (Liu et al., 2026; Zhang et al., 2025b; Xi et al., 2025). Despite this progress, most methods optimize only one fragment of the evolution process, such as retrieving experience, using it in context, distilling it into parameters, or engineering the memory architecture. Two challenges remain central: task rewards provide relatively direct supervision for execution, but not for memory selection, writing, or long-term management; and how to train these coupled abilities within one policy without making them interfere remains underexplored.

As a result, existing agents may improve within a particular setting, yet still fall short of becoming holistic agent evolvers.

We characterize a qualified agent evolver through four coupled capabilities: ❶ **experience selection** identifies useful memories from a growing and noisy repository; ❷ **experience-grounded execution** converts selected experience into effective multi-turn actions; ❸ **experience writing** extracts reusable knowledge from new trajectories and feedback; and ❹ **experience management** scores, consolidates, updates, and retires memories over time. These competencies *cannot be safely separated*: weak selection amplifies retrieval noise, weak execution leaves the agent dependent on prompt-time guidance, weak writing pollutes future memory, and weak management causes long-term degradation (Zhang et al., 2025a). This motivates our central research question:

How can we train an agent to acquire the holistic competence of evolving through experience, so as to become a qualified agent evolver?

To this end, we introduce OPD-Evolver, a slow-fast co-evolution framework where the fast loop lets the agent evolve through online interaction with environments and memory, while the slow loop distills these interaction-derived behaviors into intrinsic evolver capabilities via on-policy self-distillation. In the ♣ **fast evolution loop**, the agent operates over a four-level memory substrate of *trajectories*, *tips*, *skills*, and *tools*: it selects task-relevant memories before execution, acts with them, then writes and periodically maintains memories from trajectories, rewards, and feedback. Across this stream, OPD-Evolver estimates an *outcome-calibrated memory attribution* signal, converting delayed task outcomes into supervision for what should be selected, used, written, and preserved.

The ♠ **slow evolution loop** then turns this calibrated stream into on-policy self-distillation targets for the same policy. The privileged teacher observes attribution-enriched evidence unavailable at deployment, including candidate memory value, trajectory snippets, future utility of written memories, and repository-level diagnostics. Distilling the student’s own rollout states against this hindsight supervision jointly shapes selection, execution, writing, and management, enabling OPD-Evolver to acquire transferable lifecycle-level competence for self-improvement. Our contributions can be summarized as:

➤ **Objective Formulation.** We define the qualified agent evolver as an agent with four coupled competencies: experience selection, experience-grounded execution, experience writing, and experience management.

➤ **Practical Solution.** We propose OPD-Evolver, a fast-slow on-policy self-distillation framework that converts task outcomes and privileged hindsight into supervision for selection, execution, writing, and memory management.

➤ **Experimental Validation.** Evaluations on four self-evolving benchmarks show that OPD-Evolver-4B/9B surpass contemporary memory systems and remain competitive with giant counterparts such as QWEN3.5-397B and STEP-3.5-196B.

2 Related Work

Self-Evolving Agents. Recent self-evolving agents can be organized around the experience lifecycle defined in this work. For ❶ *experience selection*, prior systems use embedding retrieval (Tan et al., 2025), utility scoring, learned routers, or policy-based ranking (Li et al., 2026a; Xu et al., 2025) to surface useful memories from growing repositories. For ❷ *experience-grounded execution*, memory- and skill-augmented agents condition policies on retrieved memories (Ferraz et al., 2026; Wang et al., 2026d), while SFT/RL-based methods internalize experience into policy parameters (Zhang et al., 2025c; Liu et al., 2026). For ❸ *experience writing*, existing methods distill trajectories into reflections (Wu et al., 2025a; Fang et al., 2025b), reasoning memories, procedural tips (Yue et al., 2026), executable tools (Zhao et al., 2026a), or reusable skills (Xia et al., 2026; Zhang et al., 2026b). For ❹ *experience management*, recent systems study scoring, consolidation (Yue et al., 2026), forgetting, and architecture-level adaptation (Zhang et al., 2025a). These works demonstrate the value of experience reuse, but most optimize one or two stages of the lifecycle, whereas our goal is to train a truly holistic agent evolver.

On-Policy Distillation. On-policy distillation (OPD) trains a student on its own visited states by querying a teacher for dense supervision, reducing the train-inference mismatch of off-policy distillation (Hinton et al., 2015; Agarwal et al., 2024; Li et al., 2026b; Song and Zheng, 2026). Recent studies have widely explored OPD for improving agent execution, including mathematical reasoning (Wu et al., 2026), knowledge QA (Ye et al., 2026), and

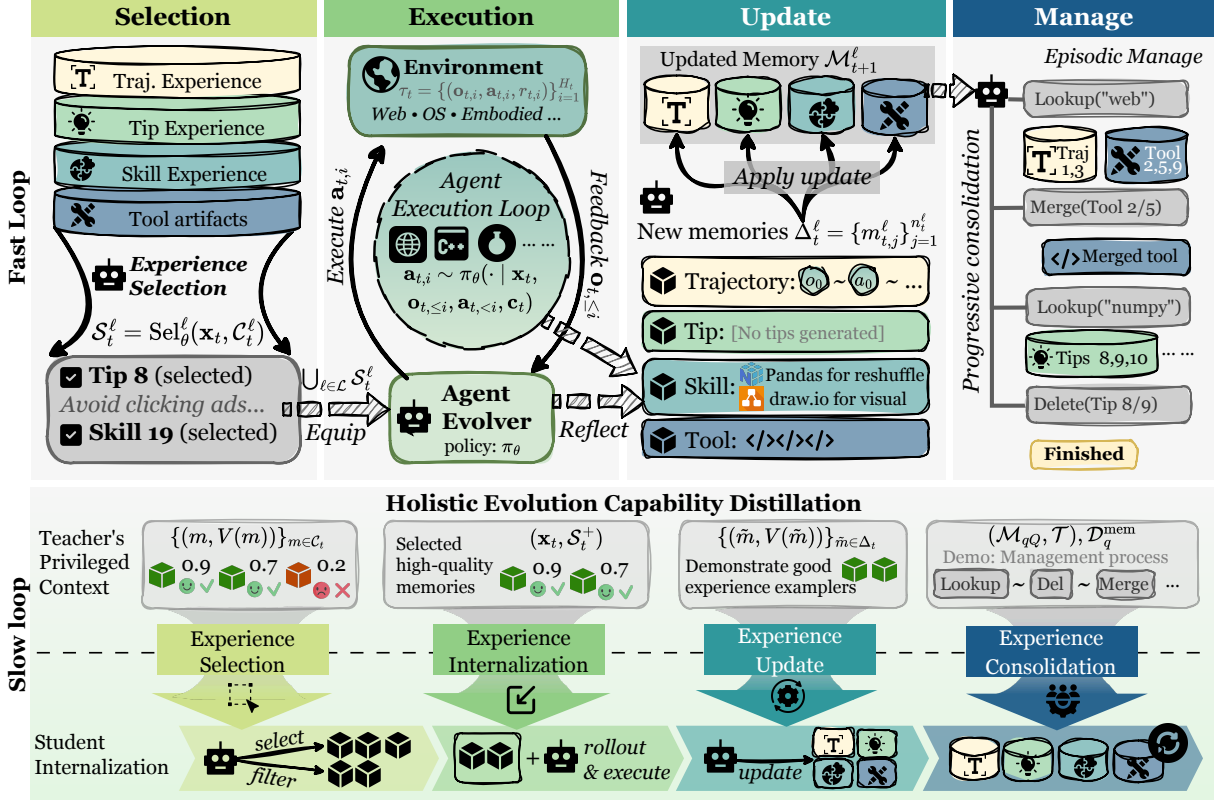


Figure 1: (Top) The **fast loop** lets the agent interact with environments and a four-level memory hierarchy; (Down) the **slow loop** converts outcome-calibrated hindsight into on-policy self-distillation signals for OPD-EvoLver.

tool-use tasks (Zhong et al., 2026). In contrast, our work uses OPD not merely to strengthen execution, but to jointly cultivate the four capabilities required by a holistic agent evolver.

3 Methodology

3.1 Problem Formulation

We consider a lifelong agent that faces a stream of interactive tasks rather than a single isolated query. At round t , an agent parameterized by θ receives a task $\mathbf{x}_t \sim \mathcal{D}$, interacts with an environment through observations $\mathbf{o}_{t,i}$ and actions $\mathbf{a}_{t,i}$, and obtains environmental feedback $r_{t,i}$. The resulting episode is denoted by

$$\tau_t = \{(\mathbf{o}_{t,i}, \mathbf{a}_{t,i}, r_{t,i})\}_{i=1}^{H_t}, R_t = \sum_{i=1}^{H_t} r_{t,i} \quad (1)$$

where H_t is the episode horizon and R_t is the task return. The per-step signal $r_{t,i}$ may be sparse; when only terminal feedback is available, we place it at the final step. Unlike standard evaluation that discards τ_t , our agent maintains an evolving repository $\mathcal{M}_t$ of reusable experience, so future behavior depends on how history is selected, used, written, and maintained. We call such an agent an **agent evolver** if it realizes a closed transition from the

current task-memory state $(\mathbf{x}_t, \mathcal{M}_t)$ to both task behavior and an updated repository. Concretely, it first retrieves a noisy candidate set $\mathcal{C}_t$ and selects a compact context $\mathcal{S}_t$:

$$\mathcal{C}_t = \text{Ret}(\mathbf{x}_t, \mathcal{M}_t), \mathcal{S}_t = \text{Sel}_\theta(\mathbf{x}_t, \mathcal{C}_t) \quad (2)$$

where Ret is a non-parametric retriever, Sel_θ is the agent’s selection behavior, and $\mathcal{S}_t \subseteq \mathcal{C}_t$. The same agent π_θ then executes with $\mathcal{S}_t$ and, after observing the outcome, emits an experience update Δ_t :

$$\tau_t \sim \pi_\theta(\cdot | \mathbf{x}_t, \mathcal{S}_t), \Delta_t \sim \pi_\theta(\cdot | \mathbf{x}_t, \tau_t, R_t, \mathcal{S}_t), \mathcal{M}_{t+1} = \mathcal{M}_t \oplus_\theta \Delta_t \quad (3)$$

where Δ_t denotes the memories and retention decisions produced from the episode, and $\oplus_\theta$ denotes the agent-controlled repository update. Thus, self-evolution depends on the full experience lifecycle, not merely a memory buffer: $\mathcal{C}_t$ must contain relevant evidence, $\mathcal{S}_t$ must be actionable, Δ_t reusable, and $\mathcal{M}_{t+1}$ useful as it grows. The streaming objective is to maximize performance along the task stream while preserving the future utility of the repository:

$$\max_\theta \liminf_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[R_t + \lambda U(\mathcal{M}_{t+1})], \quad (4)$$

where $U(\mathcal{M}_{t+1})$ denotes the downstream usefulness of the updated repository and λ controls how much the agent values future evolvability. The objective of OPD-Evolver is to train this closed loop so the agent attains high R_t while improving future repositories. The following sections instantiate this objective with a fast loop ($\triangleright$ Section 3.2) for online experience-conditioned interaction and a slow loop ($\triangleright$ Section 3.3) that distills privileged hindsight into the unified evolver.

3.2 Fast Evolution Loop

The fast loop runs online for each task and performs test-time evolution without changing θ . Its role is to turn $\mathcal{M}_t$ into a small, actionable context before execution, and to turn new episodes back into reusable experiences afterward.

Hierarchical experience memory. We organize OPD-Evolver’s $\mathcal{M}_t$ into four complementary tiers. Let $\mathcal{L} = \{\text{traj}, \text{tip}, \text{skill}, \text{tool}\}$ denote the tier set, where each $\mathcal{M}_t^\ell$ stores memories of type ℓ :

$$\mathcal{M}_t = \mathcal{M}_t^{\text{traj}} \cup \mathcal{M}_t^{\text{tip}} \cup \mathcal{M}_t^{\text{skill}} \cup \mathcal{M}_t^{\text{tool}} \quad (5)$$

where $\mathcal{M}_t^{\text{traj}}$ preserves episodic evidence, $\mathcal{M}_t^{\text{tip}}$ stores local warnings or heuristics, $\mathcal{M}_t^{\text{skill}}$ abstracts reusable procedures, and $\mathcal{M}_t^{\text{tool}}$ stores executable command or code templates. The hierarchy is useful because agent experience has different reuse granularity: full trajectories are faithful but verbose, while skills and tools are compact but must be distilled from enough evidence.

Retrieval and selection. Given the task $\mathbf{x}_t$ and optional environment metadata $\mathbf{e}_t$, we form a query $\mathbf{z}_t = [\mathbf{x}_t; \mathbf{e}_t]$ and retrieve top- K candidates from each tier by embedding similarity:

$$\mathcal{C}_t^\ell = \text{TopK}_{m \in \mathcal{M}_t^\ell} \text{sim}(\phi(\mathbf{z}_t), \phi(m)), \quad \mathcal{C}_t = \bigcup_{\ell \in \mathcal{L}} \mathcal{C}_t^\ell \quad (6)$$

where $\phi(\cdot)$ is the embedding encoder, $\text{sim}(\cdot, \cdot)$ is cosine similarity, and TopK returns the K memories with largest scores. Retrieval alone is intentionally high-recall: it surfaces potentially useful memories, but also brings stale, redundant, or task-mismatched items. The selector then compresses $\mathcal{C}_t$ into the memory context shown to the agent:

$$\mathcal{S}_t^\ell = \text{Sel}_\theta^\ell(\mathbf{x}_t, \mathcal{C}_t^\ell), \quad \mathcal{S}_t = \bigcup_{\ell \in \mathcal{L}} \mathcal{S}_t^\ell, \quad \mathbf{c}_t = \text{Fmt}(\mathcal{S}_t) \quad (7)$$

where Sel_θ^ℓ selects memories $\mathcal{S}_t^\ell$ from the tier- ℓ memory pool $\mathcal{C}_t^\ell$, and Fmt renders selected memory items into the textual prompt context. The compact context $\mathbf{c}_t$ is the operational interface between external memory and the acting policy.

Experience-grounded execution and writing.

During rollout, the actor conditions on $\mathbf{c}_t$ together with the task and interaction history:

$$\mathbf{a}_{t,i} \sim \pi_\theta(\cdot \mid \mathbf{x}_t, \mathbf{o}_{t,\leq i}, \mathbf{a}_{t,<i}, \mathbf{c}_t) \quad (8)$$

where $\mathbf{o}_{t,\leq i}$ and $\mathbf{a}_{t,<i}$ denote the partial interaction history before action $\mathbf{a}_{t,i}$. After the episode ends, the same interaction becomes evidence for future tasks. The same agent decides which tiers should be updated and how many memories to write to each tier. Given $\mathbf{x}_t$, τ_t , R_t , and $\mathcal{S}_t$, it produces $\Delta_t = \{\Delta_t^\ell\}_{\ell \in \mathcal{L}}$, where each Δ_t^ℓ may be empty or contain multiple new memories:

$$\Delta_t^\ell = \{m_{t,j}^\ell\}_{j=1}^{n_t^\ell}, \quad \mathcal{M}_{t+1}^\ell = \mathcal{M}_t^\ell \oplus_\theta \Delta_t^\ell \quad (9)$$

The count n_t^ℓ is chosen by the agent, so the fast loop can write to any subset of the four tiers rather than following a fixed memory schema. In addition to per-task writing, the agent periodically performs repository maintenance. Let $\mathcal{H}_t = \{(\tau_j, R_j, \Delta_j)\}_{j < t}$ denote the logged interaction history. Every Q tasks, it enters a multi-turn interaction with $\mathcal{H}_{qQ}$ and the current repository, using lookup plus merge(m_i, m_j) and delete(m_i):

$$\eta_q \sim \pi_\theta(\cdot \mid \mathcal{M}_{qQ}, \mathcal{H}_{qQ}, \mathcal{T}), \quad \mathcal{M}_{qQ}^+ = \mathcal{M}_{qQ} \oplus_\theta \eta_q \quad (10)$$

where q indexes maintenance rounds, qQ denotes the task index at which the q -th maintenance is triggered, $\mathcal{T} = \{\text{lookup}, \text{merge}, \text{delete}\}$, η_q is the maintenance tool-call trajectory, and $\mathcal{M}_{qQ}^+$ is the repository after maintenance. In parallel, the loop logs retrieved candidates, selected memories, outcomes, created memory ids, and maintenance actions. These logs are not needed for immediate execution, but they provide the hindsight evidence used by the slow loop to estimate memory value and distill the evolver.

3.3 Slow Evolution Loop

The fast loop lets the agent accumulate experience, but not necessarily use it well. An arbitrary agent does not know which memories help, how to ground execution in them, what deserves to become memory, or when a repository update helps future tasks. The slow loop trains these coupled abilities by turning the agent’s own interaction stream into supervision. The only trustworthy external signal available after each task is the environment feedback R_t ; the central question is how to propagate this scalar outcome back to selection, execution, writing, and maintenance decisions.

Outcome-calibrated attribution. For each memory m , we estimate its value only on tasks where m was actually retrieved, so the comparison is candidate-controlled rather than confounded by irrelevant tasks. Let $g(t)$ denote the task group of round t , and define $\Omega_g(m) = \{t : g(t) = g, m \in \mathcal{C}_t \setminus \mathcal{S}_t\}$ and $\Omega_g^+(m) = \{t : g(t) = g, m \in \mathcal{S}_t\}$. The outcome-calibrated attribution of m is

$$\hat{A}(m) = \sum_g \rho_g(m) (\mathbb{E}_{t \in \Omega_g^+(m)} [R_t] - \mathbb{E}_{t \in \Omega_g(m)} [R_t]) \quad (11)$$

where $\rho_g(m) = |\Omega_g^+(m)| / (|\Omega_g^+(m)| + |\Omega_g(m)|)$ downweights unreliable selections; empty groups are omitted. We then convert attribution into a memory score

$$\begin{aligned} V(m) &= \alpha_{\ell(m)} \gamma(m) \hat{A}(m), \\ \gamma(m) &= 1 - \frac{1}{\sqrt{1 + N^+(m)}}, \end{aligned} \quad (12)$$

where $\ell(m)$ is the tier of m , $\alpha_{\ell(m)}$ is a tier prior, $N^+(m) = |\Omega_g^+(m)|$, and $\gamma(m)$ is a confidence factor. This calibration converts delayed environment feedback into dense hindsight labels: memories with high $V(m)$ become evidence for what should have been selected, used, and preserved, while low-value memories expose noise that the evolver should learn to ignore.

Unified hindsight self-distillation. We use the calibrated stream to train the same agentic evolver under a single on-policy distillation principle. Let $\mathcal{K} = \{\text{sel}, \text{act}, \text{write}, \text{maint}\}$. For each lifecycle decision k , the student sees only the public input z^k , while the teacher additionally sees a privileged hindsight view h^k :

$$\begin{aligned} (z_t^{\text{sel}}, h_t^{\text{sel}}) &= ((\mathbf{x}_t, \mathcal{C}_t), \{(m, V(m))\}_{m \in \mathcal{C}_t}), \\ (z_t^{\text{act}}, h_t^{\text{act}}) &= (\mathbf{x}_t, (\mathcal{S}_t^+, \tau_t^+)), \\ (z_t^{\text{write}}, h_t^{\text{write}}) &= ((\mathbf{x}_t, \tau_t, R_t, \mathcal{S}_t), \{(\tilde{m}, V(\tilde{m}))\}_{\tilde{m} \in \Delta_t}), \\ (z_q^{\text{maint}}, h_q^{\text{maint}}) &= ((\mathcal{M}_{qQ}, \mathcal{H}_{qQ}, \mathcal{T}), \mathcal{D}_q^{\text{mem}}) \end{aligned} \quad (13)$$

where $\mathcal{S}_t^+ = \{m \in \mathcal{S}_t : V(m) > 0\}$ denotes valuable selected memories, τ_t^+ is a successful trajectory from the same task group, and $\mathcal{D}_q^{\text{mem}} = \{(m, V(m), \gamma(m), \nu(m))\}_{m \in \mathcal{M}_{qQ}} \cup \{\kappa(m_i, m_j)\}_{i,j}$ contains memory value, confidence, usage statistic $\nu(m)$, and redundancy score $\kappa(m_i, m_j)$. Thus, ❶ selection sees the value of every retrieved candidate; ❷ execution internalizes useful selected memories by asking the student to solve without memory; ❸ writing sees which produced memories later become valuable; and ❹

maintenance sees calibrated diagnostics for producing merge/delete tool-call trajectories. For each (z^k, h^k) , the student first samples an on-policy output $\hat{\mathbf{y}}^k$ under the deployment condition, and the teacher is evaluated on the same student prefixes:

$$\begin{aligned} \hat{\mathbf{y}}^k &\sim p_\theta^S(\cdot | z^k), \quad p_{\theta,n}^S = p_\theta^S(\cdot | z^k, \hat{\mathbf{y}}_{<n}^k), \\ p_{\theta,n}^T &= p_\theta^T(\cdot | z^k, h^k, \hat{\mathbf{y}}_{<n}^k) \end{aligned} \quad (14)$$

where p_θ^S is the student policy, p_θ^T is the same evolver under privileged conditioning with stop-gradient parameters θ , and $\hat{\mathbf{y}}_{<n}^k$ is the student-generated prefix. Let $\delta_{k,n}$ denote the corresponding token-level discrepancy:

$$\delta_{k,n} = D_{\text{tok}}(\text{sg}[p_{\theta,n}^T] \parallel p_{\theta,n}^S) \quad (15)$$

where D_{tok} is full-vocabulary token-level KL divergence, and $\text{sg}[\cdot]$ blocks gradients through the teacher distribution. The unified slow-loop objective is the token-level distillation loss over student-visited prefixes:

$$\mathcal{L}_{\text{slow}}(\theta) = \sum_{k \in \mathcal{K}} \mathbb{E}_{\substack{(z^k, h^k) \sim d_{\pi_\theta}^k \\ \hat{\mathbf{y}}^k \sim p_\theta^S(\cdot | z^k)}} \left[\frac{1}{L_k} \sum_{n=1}^{L_k} \delta_{k,n} \right] \quad (16)$$

where $d_{\pi_\theta}^k$ is the on-policy distribution of decision inputs for lifecycle decision k . Selection, execution, writing, and maintenance are therefore supervised as four views of one experience lifecycle: the teacher explains what the agent should have selected, how it should have acted with useful experience, and what it should have written or retained after observing the outcome. After distillation, only the student-facing behavior is deployed back into the fast loop, so the agent can exercise these evolver abilities without privileged feedback at test time.

4 Experiments

4.1 Experiment Setup

Training setup. We train OPD-Evolver on heterogeneous agentic experience from Agent World Model (Wang et al., 2026c), nvidia/Nemotron-Terminal-Corpus (Pi et al., 2026), and EnvScaler (Song et al., 2026), which are disjoint from all evaluation benchmarks (see Section D). Unless specified otherwise, we use QWEN3-4B-INSTRUCT-2507 (Yang et al., 2025) and QWEN3.5-9B (Qwen, 2026) as backbones, QWEN3-EMBEDDING-0.6B for retrieval, retrieve 50 candidates, set $Q = 30$, keep at most 20 memories in privileged teacher contexts, filter supervision with minimum score 0.01. More parameter setups are detailed in Section D.

Table 1: Main results across self-evolving agent benchmarks. Each cell reports the official task success metric for the corresponding benchmark subset; higher is better. For AMA-Bench, CI: Causal Inference, SU: State Updating, SA: State Updating. **Best** and **second-best** mark the top two results within each backbone group.

LLM	Method	LifelongAgentBench		MemoryArena		AMA-Bench			InterCode		
		DB	OS	Math	Physics	CI	SU	SA	Bash	CTF	SQL
QWEN3.5-397B-A17B		86.00	63.00	3.98	4.65	57.21	58.73	51.41	51.34	56.00	62.74
STEP-3.5-FLASH (196B)		81.00	58.00	1.98	2.33	49.50	46.99	48.88	44.20	48.00	59.87
QWEN3-4B	No Memory	65.00	36.50	2.40	2.33	24.83	27.67	29.73	30.36	26.00	38.85
	ExpeL	62.50	34.50	1.98	1.74	26.17	24.88	28.49	30.80	33.00	34.08
	AWM	63.00	36.00	1.69	3.49	28.36	30.14	33.49	27.68	33.00	38.54
	Cheatsheet	69.00	38.50	2.97	1.74	25.34	30.91	30.37	33.04	25.00	37.90
	Memp	73.00	41.00	4.66	0.00	32.38	28.44	31.81	31.70	27.00	43.31
	ReasoningBank	70.00	38.00	3.81	1.16	27.35	29.68	30.97	31.25	25.00	43.95
	EvolveR	71.50	46.50	1.27	2.91	26.51	24.42	30.01	33.93	31.00	44.90
	MemEvolve	72.00	44.00	1.84	2.33	29.53	31.53	32.73	33.93	29.00	44.59
	OPD-Evolver-4B	74.00	49.50	5.51	4.07	32.89	34.93	34.90	36.16	34.00	45.86
QWEN3.5-9B	No Memory	75.50	52.50	5.51	5.23	40.10	47.14	45.63	41.52	44.00	55.73
	ExpeL	74.00	46.50	8.19	8.72	41.11	43.74	44.67	40.18	50.00	52.23
	AWM	72.50	44.00	7.34	11.05	43.96	48.38	48.00	39.73	49.00	51.59
	Cheatsheet	79.50	52.50	6.64	5.23	40.77	49.30	46.19	41.96	47.00	56.05
	Memp	82.00	56.00	4.80	6.40	45.81	47.14	46.59	44.20	50.00	58.28
	ReasoningBank	80.50	55.00	4.94	6.98	42.28	48.38	47.04	43.75	45.00	57.96
	EvolveR	82.50	59.50	6.07	7.56	41.78	42.81	46.11	44.64	52.00	60.51
	MemEvolve	81.00	61.00	4.24	9.88	44.30	49.77	47.20	45.98	53.00	61.15
	OPD-Evolver-9B	84.50	65.00	10.88	11.63	47.32	53.94	52.92	49.55	57.00	64.01

Algorithm 1: Workflow of OPD-Evolver.

```

Input: Task stream  $\{\mathbf{x}_t\}$ ; memory repository  $\mathcal{M}_0$ ;
        evolver policy  $\pi_\theta$ ; maintenance period  $Q$ 
Output: Deployable evolver  $\pi_\theta^S$  and evolved
        repository  $\mathcal{M}$ 
/* Fast loop: online experience use */
1 for each task  $\mathbf{x}_t$  do
2   Form query  $\mathbf{z}_t$  from task and environment
3   for  $\ell \in \mathcal{L}$  do
4     Retrieve  $\mathcal{C}_t^\ell$  from  $\mathcal{M}_t^\ell$  and select  $\mathcal{S}_t^\ell \subseteq \mathcal{C}_t^\ell$ 
5    $\mathcal{S}_t \leftarrow \bigcup_{\ell \in \mathcal{L}} \mathcal{S}_t^\ell$ ,  $\mathbf{c}_t \leftarrow \text{Fmt}(\mathcal{S}_t)$ 
6   Roll out  $\pi_\theta$  with  $\mathbf{c}_t$  to obtain  $(\tau_t, R_t)$ 
7   for  $\ell \in \mathcal{L}$  do
8     Write  $\Delta_t^\ell$  and update  $\mathcal{M}_{t+1}^\ell \leftarrow \mathcal{M}_t^\ell \oplus_\theta \Delta_t^\ell$ 
9   if  $t \bmod Q = 0$  then
10     $q \leftarrow t/Q$ 
11    Produce maintenance trajectory  $\eta_q$  with
        {lookup, merge, delete} and update
         $\mathcal{M}_{qQ}^+$ 
12    Log  $(\mathcal{C}_t, \mathcal{S}_t, R_t, \Delta_t)$  and any  $\eta_q$ 
/* Slow loop: unified distill */
13 for each logged memory  $m$  do
14   Estimate  $V(m)$  from logged outcomes
15   for  $k \in \{\text{sel}, \text{act}, \text{write}, \text{maint}\}$  do
16    Construct  $(z^k, h^k)$ , sample  $\hat{\mathbf{y}}^k \sim p_\theta^S(\cdot | z^k)$ ,
        and minimize  $\mathcal{L}_{\text{slow}} // \text{OPD}$ 
17 return Qualified agent evolver  $\pi_\theta^S$ 

```

Baselines. We compare with **memory-augmented agents:** ExpeL (Zhao et al., 2024), AWM (Wang et al., 2024), Cheatsheet (Suzgun et al., 2025), Memp (Fang et al., 2025c), ReasoningBank (Ouyang et al., 2025), EvolveR (Wu et al., 2025a), and MemEvolve (Zhang et al.,

2025a); and **training-based methods:** SFT, GRPO (Shao et al., 2024), Skill0 (Lu et al., 2026), MemRL (Zhang et al., 2026c) and Complementary RL (Muhtar et al., 2026). For fairness, all memory-based methods, including OPD-Evolver, start evaluation with an empty experience repository and accumulate memory only from the evaluation stream (see details in Section C.1).

Evaluation benchmarks. We evaluate on five benchmarks: **LifelongAgentBench** (Zheng et al., 2025b) with *database (DB)* and *operating system (OS)*; **MemoryArena** (He et al., 2026) with *Math* and *Physics* subsets; **AMA-Bench** (Zhao et al., 2026b) with *Causal Inference*, *State Updating*, and *State Abstraction*; **InterCode** (Yang et al., 2023) with *Bash*, *CTF*, and *SQL* subsets; and embodied environment **MiniHack** (Samvelyan et al., 2021). Benchmark details are in Section C.

4.2 Main Results

Comparison with Memory Systems We first compare OPD-Evolver with seven mainstream self-evolving memory systems as well as several larger models in Table 1. OPD-Evolver achieves the best result on all 10 subsets among same-backbone memory baselines for both 4B and 9B, with clear gains over the strongest memory baseline on OS (65.00% vs. 61.00%), AMA-SA (52.92% vs. 48.00%), and InterCode-CTF (57.00% vs. 53.00%) for 9B. Notably, OPD-Evolver-9B challenges its

Table 2: Comparison with training-based agent improvement methods. Each cell reports the official task success metric.

Method	MiniHack			InterCode		
	Room	Maze	KeyRoom	Bash	CTF	SQL
Vanilla	80.39	19.61	0.00	55.73	44.00	41.25
SFT	82.35	17.65	0.00	59.87	49.00	44.64
GRPO	100.00	23.53	3.92	63.69	58.00	47.77
Skill0	94.12	25.49	3.92	62.10	54.00	47.32
MemRL	96.08	19.61	1.96	61.15	50.00	44.20
Complementary RL	96.88	20.85	5.16	63.20	55.00	48.10
OPD-Evolver	98.04	27.45	9.80	64.01	59.00	49.55

much larger counterparts: OPD-Evolver-9B exceeds STEP-3.5-FLASH (196B) on 9/10 subsets and surpasses QWEN3.5-397B-A17B on 6/10 subsets, including AMA-SA (52.92% vs. 51.41%), InterCode-CTF (57.00% vs. 56.00%), and SQL (64.01% vs. 62.74%), showing that lifecycle-level evolution can make a compact agent competitive with far larger counterparts.

Comparison with Training-based Methods Table 2 shows that OPD-Evolver achieves the best average performance among training-based methods, winning on 5/6 subsets. Compared with GRPO, OPD-Evolver improves Maze (27.45% vs. 23.53%), KeyRoom (9.80% vs. 3.92%), Bash (64.01% vs. 63.69%), CTF (59.00% vs. 58.00%), and SQL (49.55% vs. 47.77%). Against Complementary RL, OPD-Evolver further improves the hardest MiniHack subsets, including Maze (27.45% vs. 20.85%) and KeyRoom (9.80% vs. 5.16%). It also consistently outperforms MemRL on both MiniHack and InterCode, with a 5.35-point gain on SQL. These results suggest that OPD-Evolver learns beyond task-level reward fitting, yielding a more transferable mechanism for selecting, internalizing, and reusing experience.

4.3 Ablation Study

We ablate five components of OPD-Evolver in Table 3: ❶ w/o Slow Evolution removes on-policy distillation, ❷ w/o Memory Attribution replaces outcome-calibrated attribution with a simpler memory-frequency calibrated score, ❸ w/o Selection directly chooses the top-5 memories by similarity, ❹ w/o Writing Distillation removes supervision for reusable memory writing, and ❺ w/o Maintenance disables repository update training. All variants degrade InterCode performance, with the largest drop from removing memory attribution (average 38.67% $\rightarrow$ 32.13%; Bash/CTF/SQL decrease by 4.96%, 7.31%, and 7.36%). Removing slow evolution is similarly harmful (average

Table 3: Ablation study on InterCode (Bash/CTF/SQL) with OPD-Evolver-4B. “Writing Distill.” denotes the exclusion of self-distilling experience writing capability.

Variant	Bash	CTF	SQL
OPD-Evolver-4B	36.16	34.00	45.86
w/o Slow Evolution	32.14	28.00	39.17
w/o Memory Attribution	31.20	26.69	38.50
w/o Selection	35.27	32.00	42.04
w/o Writing Distill.	34.38	29.00	41.08
w/o Maintenance	35.30	30.10	43.51

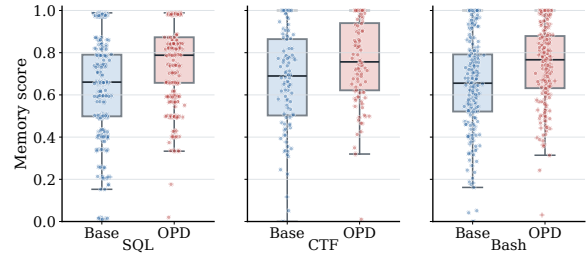


Figure 2: Distribution of calibrated memory scores for memories selected by the original QWEN3.5-9B and OPD-Evolver-9B after selection distillation.

33.10%), showing that privileged hindsight must be distilled into the deployable policy. Selection, writing, and maintenance also matter: similarity-only selection reduces SQL from 45.86% to 42.04%, and removing writing distillation lowers CTF from 34.00% to 29.00%. Together, these results show that calibrated attribution, learned selection, memory writing, and repository maintenance must be trained jointly for reliable self-evolution.

4.4 Framework Analysis

Impact of Selection Distillation. We compare the memory scores of items selected by the original 9B backbone and the trained OPD-Evolver-9B selector. As shown in Figure 2, selection distillation consistently raises the median selected-memory score, from 0.66/0.69/0.66 to 0.79/0.76/0.76 on SQL/CTF/Bash. The lower quartile also moves upward from 0.50 to above 0.62 across the three subsets, suggesting that OPD-Evolver reduces low-utility retrieval noise rather than merely shifting a few high-scoring outliers.

Impact of Writing Distillation We next examine whether the trained evolver writes memories with higher future utility. Figure 3 compares the calibrated score distribution of memories produced by vanilla QWEN3.5-9B and by OPD-Evolver-9B. Writing distillation increases the median score from 0.80/0.82/0.82 to 0.91/0.90/0.89 on

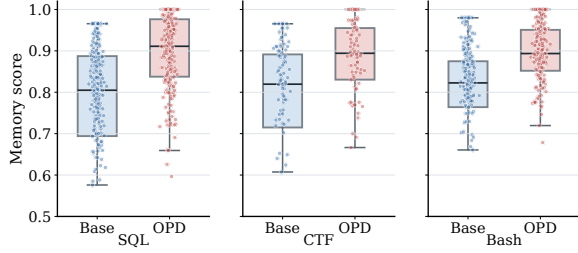


Figure 3: Distribution of calibrated memory scores for memories written by the original QWEN3.5-9B and OPD-Evolver-9B after writing distillation.

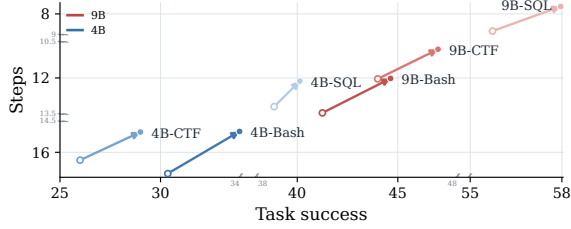


Figure 4: Experience internalization effect. Each arrow starts from the vanilla backbone and ends at the corresponding OPD-Evolver policy; rightward movement indicates higher task success and upward movement in the inverted axis indicates fewer steps.

SQL/CTF/Bash, and lifts the lower quartile to 0.83 or higher. This tighter high-score distribution indicates that OPD-Evolver writes memories that are not only better formatted, but more reliably useful for downstream tasks.

Impact of Experience Internalization Finally, we test whether slow-loop distillation internalizes useful experience into the policy. Figure 4 compares each vanilla backbone with its trained OPD-Evolver counterpart (w/o external $\mathcal{M}$; merely for task execution) in terms of task success and execution steps. Every arrow moves toward *higher success* and *fewer steps*: OPD-Evolver raises success by about 3–4 points for 4B and 3–7 points for 9B on Bash/CTF/SQL, while reducing trajectories by up to 2.5 steps. This suggests that OPD-Evolver converts high-value memories into more direct and efficient behavior, rather than merely retrieving better context at inference time.

4.5 Case Study

Figure 5 illustrates how slow-loop training makes both memory selection and memory writing more precise. For the LifelongAgentBench-OS task, the vanilla model selects broad directory and configuration memories, while OPD-Evolver keeps only the modification-log skill and permission tip that directly support creating files, setting modes, and

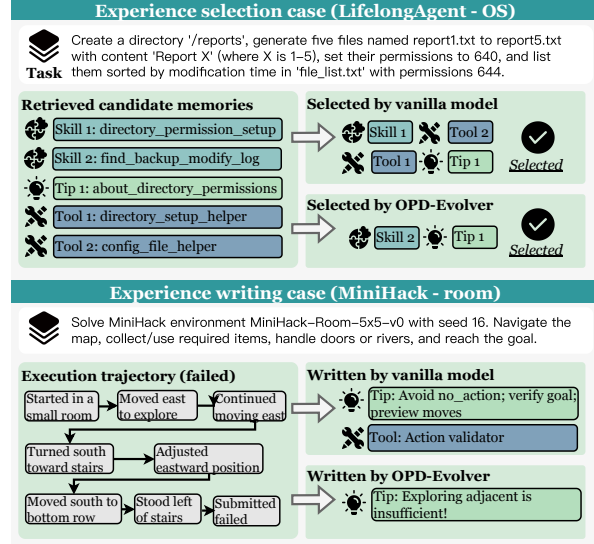


Figure 5: Case study of experience selection and writing in OPD-Evolver. (**Top**) on LifelongAgentBench-OS, OPD-Evolver selects a compact task-matched subset from retrieved memories. (**Bottom**) on MiniHack-Room, OPD-Evolver writes a more causal and reusable memory from a failed trajectory.

producing the sorted file list. For the MiniHack-Room failure, the vanilla model writes generic advice such as verifying the goal and adding an action validator, but the trajectory shows a sharper error: the agent stopped after local east/south exploration without sufficiently expanding adjacent states around the stairs. OPD-Evolver therefore writes the reusable causal tip “Exploring adjacent is insufficient,” showing that the trained evolver learns not only to retrieve relevant experience, but also to turn failures into compact future-facing memory. More case studies are at Section F.

5 Conclusion

In this work, we present OPD-Evolver, a slow-fast co-evolution framework that combines online memory-conditioned interaction with outcome-calibrated attribution and privileged on-policy self-distillation. Across diverse evaluation benchmarks, OPD-Evolver consistently improves compact open models over memory-augmented and training-based baselines, showing that evolver abilities learned from heterogeneous streams can transfer beyond the training environments. Together, these results suggest a shift from building agents that merely store experience toward cultivating agents that can continuously transform experience into their own capacity for evolution.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2024. **On-policy distillation of language models: Learning from self-generated mistakes**. *Preprint*, arXiv:2306.13649.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025a. **A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems**. *Preprint*, arXiv:2508.07407.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2025b. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2025c. **Memp: Exploring agent procedural memory**. *Preprint*, arXiv:2508.06433.
- Thomas Palmeira Ferraz, Romain Deffayet, Vassilina Nikoulina, Hervé Déjean, and Stéphane Clinchant. 2026. **Retrieval-augmented llm agents: Learning to learn from experience**. *Preprint*, arXiv:2603.18272.
- Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. 2026. **Memo-ryarena: Benchmarking agent memory in inter-dependent multi-session agentic tasks**. *Preprint*, arXiv:2602.16313.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. **Distilling the knowledge in a neural network**. *Preprint*, arXiv:1503.02531.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, and 28 others. 2025. **Memory in the age of ai agents**. *Preprint*, arXiv:2512.13564.
- Chunyu Li, Jingyi Kang, Ding Chen, Mengyuan Zhang, Jiajun Shen, Bo Tang, Xuanhe Zhou, Feiyu Xiong, and Zhiyu Li. 2026a. **Memreranker: Reasoning-aware reranking for agent memory retrieval**. *Preprint*, arXiv:2605.06132.
- Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. 2026b. **Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe**. *Preprint*, arXiv:2604.13016.
- Xuechen Liang, Meiling Tao, Yinghui Xia, Tianyu Shi, Jun Wang, and JingSong Yang. 2024. Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872*.
- Weize Liu, Minghui Liu, Sy-Tuyen Ho, Souradip Chakraborty, Xiyao Wang, and Furong Huang. 2026. **Agentic critical training**. *Preprint*, arXiv:2603.08706.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*.
- MiniMax. 2026. MiniMax M2.7 - Model Self-Improvement, Driving Productivity Innovation Through Technological Breakthroughs — minimax.io. <https://www.minimax.io/models/text/m27>.
- Dilxat Muhtar, Jiashun Liu, Wei Gao, Weixun Wang, Shaopan Xiong, Ju Huang, Siran Yang, Wenbo Su, Jiamang Wang, Ling Pan, and 1 others. 2026. Complementary reinforcement learning. *arXiv preprint arXiv:2603.17621*.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjuan Jiang. 2026. **Trace2skill: Distill trajectory-local lessons into transferable agent skills**. *Preprint*, arXiv:2603.25158.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2025. **Reasoningbank: Scaling agent self-evolving with reasoning memory**. *Preprint*, arXiv:2509.25140.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. **Natural-language agent harnesses**. *Preprint*, arXiv:2603.25723.
- Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Janaty, Bryan Catanzaro, and Wei Ping. 2026. **On data engineering for scaling llm terminal capabilities**. *Preprint*, arXiv:2602.21193.
- Qwen. 2026. Qwen Studio — qwen.ai. <https://qwen.ai/blog?id=qwen3.5>.
- Mikayel Samvelyan, Robert Kirk, Vitaly Kurin, Jack Parker-Holder, Minqi Jiang, Eric Hambro, Fabio Petroni, Heinrich Küttler, Edward Grefenstette, and Tim Rocktäschel. 2021. **Minihack the planet: A sandbox for open-ended reinforcement learning research**. *Preprint*, arXiv:2109.13202.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Junhao Shen, Teng Zhang, Xiaoyan Zhao, and Hong Cheng. 2026. **Dynamic skill lifecycle management for agentic reinforcement learning**. *Preprint*, arXiv:2605.10923.
- Yaorui Shi, Yuxin Chen, Zhengxi Lu, Yuchun Miao, Shugui Liu, Qi GU, Xunliang Cai, Xiang Wang, and An Zhang. 2026. **Skill1: Unified evolution of skill-augmented agents via reinforcement learning**. *Preprint*, arXiv:2605.06130.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.
- Mingyang Song and Mao Zheng. 2026. **A survey of on-policy distillation for large language models**. *Preprint*, arXiv:2604.00626.
- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2026. **Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis**. *Preprint*, arXiv:2601.05808.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2025. **Dynamic cheat-sheet: Test-time learning with adaptive memory**. *Preprint*, arXiv:2504.07952.
- Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long T. Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. 2025. **In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents**. *Preprint*, arXiv:2503.08026.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, and 150 others. 2025. **Kimi k2: Open agentic intelligence**. *Preprint*, arXiv:2507.20534.
- Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A. Rossi, Jingbo Shang, Julian McAuley, and Junda Wu. 2026. **Skill-r1: Agent skill evolution via reinforcement learning**. *Preprint*, arXiv:2605.09359.
- Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. 2026a. **Skill-sd: Skill-conditioned self-distillation for multi-turn llm agents**. *Preprint*, arXiv:2604.10674.
- Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. 2026b. **Openclaw-rl: Train any agent simply by talking**. *Preprint*, arXiv:2603.10165.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. 2026c. **Agent world model: Infinity synthetic environments for agentic reinforcement learning**. *Preprint*, arXiv:2602.10090.
- Zhenting Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. 2026d. **Memex(rl): Scaling long-horizon llm agents via indexed experience memory**. *Preprint*, arXiv:2603.04257.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. **Agent workflow memory**. *Preprint*, arXiv:2409.07429.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, and 1 others. 2025. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*.
- Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung yi Lee. 2024. **Streambench: Towards benchmarking continuous improvement of language agents**. *Preprint*, arXiv:2406.08747.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xue-meng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2025a. **Evolver: Self-evolving llm agents through an experience-driven lifecycle**. *Preprint*, arXiv:2510.16079.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xue-meng Yang, Yufan Shen, Yuxin Wang, and 1 others. 2025b. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*.
- Yecheng Wu, Song Han, and Hai Cai. 2026. **Lightning opd: Efficient post-training for large reasoning models with offline on-policy distillation**. *Preprint*, arXiv:2604.13010.
- Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, Wei He, Yiwen Ding, Guanyu Li, Zehui Chen, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Tao Gui, and 4 others. 2025. **Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning**. *Preprint*, arXiv:2509.08755.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, and 1 others. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.

- Derong Xu, Yi Wen, Pengyue Jia, Yingyi Zhang, wenlin zhang, Yichao Wang, Huifeng Guo, Ruiming Tang, Xiangyu Zhao, Enhong Chen, and Tong Xu. 2025. **From single to multi-granularity: Toward long-term memory association and selection of conversational agents**. *Preprint*, arXiv:2505.19549.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. **Qwen3 technical report**. *Preprint*, arXiv:2505.09388.
- John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. **Intercode: Standardizing and benchmarking interactive coding with execution feedback**. *Preprint*, arXiv:2306.14898.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. 2026. **On-policy context distillation for language models**. *Preprint*, arXiv:2602.12275.
- Yanwei Yue, Boci Peng, Xuanbo Fan, Jiaxin Guo, Qiankun Li, and Yan Zhang. 2026. **Mem-t: Den-sifying rewards for long-horizon memory agents**. *Preprint*, arXiv:2601.23014.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. 2025a. **Memevolve: Meta-evolution of agent memory systems**. *Preprint*, arXiv:2512.18746.
- Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Jat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, Jie Tang, and Yuxiao Dong. 2025b. **Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework**. *Preprint*, arXiv:2510.04206.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026a. **Memskill: Learning and evolving memory skills for self-evolving agents**. *arXiv preprint arXiv:2602.02474*.
- Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, Jian Xie, Yuxuan Sun, Boyu Gou, Qi Qi, Zihang Meng, Jianwei Yang, Ning Zhang, Xian Li, Ashish Shah, and 11 others. 2025c. **Agent learning via early experience**. *Preprint*, arXiv:2510.08558.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. 2026b. **Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory**. *Preprint*, arXiv:2601.03192.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, and 1 others. 2026c. **Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory**. *arXiv preprint arXiv:2601.03192*.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. **Expel: Llm agents are experiential learners**. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.
- Shitian Zhao, Shaoheng Lin, Ming Li, Haoquan Zhang, Wenshuo Peng, Kaipeng Zhang, and Chen Wei. 2026a. **Pyvision-rl: Forging open agentic vision models via rl**. *Preprint*, arXiv:2602.20739.
- Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuan-dong Tian, and Jishen Zhao. 2026b. **Ama-bench: Evaluating long-horizon memory for agentic applications**. *Preprint*, arXiv:2602.22769.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025a. **Skillweaver: Web agents can self-improve by discovering and honing skills**. *Preprint*, arXiv:2504.07079.
- Junhao Zheng, Xidi Cai, Qiuqi Li, Duzhen Zhang, Zhongzhi Li, Yingying Zhang, Le Song, and Qianli Ma. 2025b. **Lifelongagentbench: Evaluating llm agents as lifelong learners**. *Preprint*, arXiv:2505.11942.
- Qiyong Zhong, Mao Zheng, Mingyang Song, Xin Lin, Jie Sun, Houcheng Jiang, Xiang Wang, and Jun-feng Fang. 2026. **Sod: Step-wise on-policy distillation for small language model agents**. *Preprint*, arXiv:2605.07725.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, and 2 others. 2026. **Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering**. *Preprint*, arXiv:2604.08224.

A Related Work Discussion

We further discuss several closely related contemporary works, many of which study skill-based experience internalization. SKILL0 (Lu et al., 2026) and OpenClaw-RL (Wang et al., 2026b) use RL or OPD to internalize agent execution behaviors, while our focus additionally includes how experience is selected, written, and maintained. SkillRL (Xia et al., 2026), Trace2Skill (Ni et al., 2026), and Skill-R1 (Vishe et al., 2026) mainly

improve skill or lesson generation, corresponding most closely to the experience-writing part of our lifecycle. Skill-SD (Wang et al., 2026a) uses skills as privileged context during distillation, emphasizing experience-guided execution. SLIM (Shen et al., 2026) and Skill1 (Shi et al., 2026) take broader views of skill lifecycle management. In comparison, OPD-Evolver studies a unified evolver policy over multiple experience forms, including trajectories, tips, skills, and tools, and evaluates its cross-domain generalization across heterogeneous agent benchmarks.

B Potential Risks

We do not observe direct evidence that OPD-Evolver introduces qualitatively new social risks beyond those already associated with LLM-based agents. However, because the framework improves an agent’s ability to reuse experience and maintain memory, standard concerns around unsafe tool use, privacy leakage, biased behavior, and unintended task generalization still require careful attention. In practice, deployments should follow the same safeguards expected for agentic LLM systems, including controlled environments, memory auditing, access restrictions, and human oversight when actions may affect real users or external systems.

C Benchmark Details

This section provides additional details on the benchmarks used for evaluation, including their subsets, test-set sizes, environment interfaces, and action spaces.

LifelongAgentBench. LifelongAgentBench (Zheng et al., 2025b) is a multi-domain lifelong agent benchmark. We evaluate on its database (DB) and operating system (OS) subsets, with 100 test tasks for each subset. The DB subset consists of MySQL tasks, where the agent receives a natural-language instruction together with database context and acts through SQL execution followed by final submission. The OS subset consists of Ubuntu-container tasks, where the agent receives a natural-language instruction and interacts with the environment through bash commands followed by final submission. These two subsets evaluate whether the agent can reuse memory for long-horizon executable tasks that require precise environment operations.

MemoryArena. MemoryArena (He et al., 2026) evaluates memory over multi-session reasoning tasks. We evaluate on the Math and Physics subsets, which contain 354 and 86 test tasks, respectively. Each task provides shared background information and a sequence of question-answer sessions. The agent must answer the current question while preserving and reusing information from prior sessions. Unlike interactive control benchmarks, MemoryArena does not require live environment actions; instead, it tests whether the agent can maintain, retrieve, and apply useful memory across sessions.

AMA-Bench. AMA-Bench (Zhao et al., 2026b) evaluates long-horizon memory over completed agent trajectories. We use 208 episodes, containing 2,496 question-answer pairs in total. Following the benchmark taxonomy, the QA pairs are divided into four categories: Recall (839 pairs), Causal Inference (596 pairs), State Updating (647 pairs), and State Abstraction (414 pairs). In our evaluation, we focus on the memory-intensive Causal Inference, State Updating, and State Abstraction categories. Each example contains an action-observation trajectory and a question about the trajectory. Causal Inference requires the agent to reason about why events occurred, State Updating requires tracking changes across the trajectory, and State Abstraction requires summarizing or abstracting the relevant state. The agent does not control a live environment in AMA-Bench; it reads the trajectory and produces an answer, optionally using retrieved memory.

InterCode. InterCode (Yang et al., 2023) provides interactive coding benchmarks in isolated execution environments. We evaluate on the SQL, Bash, and CTF subsets, which contain 314, 224, and 100 test tasks, respectively. In SQL, the agent solves text-to-SQL tasks by issuing SQL queries against a database. In Bash, the agent solves natural-language shell tasks by executing bash commands. In CTF, the agent solves capture-the-flag challenges by executing commands and attempting flags. In all three subsets, the agent interacts through a JSON action interface and terminates with final submission. For Bash and SQL, grading is based on the output of the last executed command or query.

MiniHack. MiniHack (Samvelyan et al., 2021) is a suite of procedurally generated grid-world

navigation environments built on NetHack. We evaluate on Room, Maze, and KeyRoom, with 51 test tasks for each environment. Room requires the agent to reach the goal in a small open room; Maze requires systematic exploration in a larger maze; and KeyRoom requires the agent to locate a key, open a locked door, and then reach the goal. We render each environment as a text-based dungeon view using NetHack glyph conventions, such as @ for the agent, > for stairs or the goal, and . for floor tiles. At each step, the agent receives the ASCII map, the latest environment message, inventory text, and step/reward statistics. The agent acts through a unified JSON action interface with compass moves, object-interaction actions such as pickup, apply, open, and search, navigation actions such as climb_up and wait, and final submit.

C.1 Evaluation Protocol

For all memory-based methods, evaluation starts from an empty experience repository $\mathcal{M}_0 = \emptyset$. The agent then solves tasks sequentially in a streaming order, so its repository can grow only from trajectories, memories, and feedback observed during the current evaluation stream. After each task, the environment returns task-level feedback indicating whether the attempt succeeds or fails, which is used for memory update and attribution. The agent does not receive the ground-truth answer or an expert solution after the task; it only observes the same environment feedback available to all compared methods. For the training-based comparison in Table 2, we additionally train all compared methods on MiniHack using the same training data and evaluate them on the same test tasks to ensure a fair comparison.

D Training Details

D.1 Training Data

We construct the training data from 7,000 interactive agent tasks: 3,000 from Agent World Model (AWM) (Wang et al., 2026c), 2,000 from nvidia/Nemotron-Terminal-Corpus (Pi et al., 2026), and 2,000 from EnvScaler (Song et al., 2026). These sources cover diverse executable and tool-interactive scenarios, exposing OPD-Evolver to heterogeneous trajectories for memory selection, memory writing, memory-conditioned execution, and repository maintenance.

Agent World Model (AWM). AWM provides synthetic multi-turn tool-use environments with executable, code-driven state transitions and database-backed application scenarios. We use it to collect trajectories with reliable task-level outcomes and rich tool interactions.

nvidia/Nemotron-Terminal-Corpus.

Nemotron-Terminal-Corpus provides terminal-style interaction data covering command execution, shell operations, and multi-step problem solving in textual environments. It complements AWM by exposing the agent to command-line workflows and environment feedback patterns.

EnvScaler. EnvScaler provides scalable tool-interactive environments for collecting diverse agent trajectories. We use it to broaden the training distribution beyond a single benchmark style and to expose OPD-Evolver to varied interaction structures.

Note that, for the training-based comparison in Table 2, we additionally train all compared methods on MiniHack using the same training data and evaluate them on the same test tasks to ensure a fair comparison.

D.2 Training Parameters

This section summarizes the training and rollout hyperparameters used in our experiments.

Shared Settings Unless otherwise specified, we use Qwen3.5-9B and Qwen3-4B backbones with bf16 precision. We set the maximum prompt length to 8192 tokens. For GRPO training, we use temperature 1.0 and top- p 1.0. For OPD-Evolver distillation data generation, we use temperature 1.0 and top- p 0.95. Across interactive environments, the maximum episode length is 40 steps unless otherwise specified.

SFT For the SFT baseline, we fine-tune the base model with LoRA. We use a learning rate of 1×10^{-5} , per-device batch size 2, gradient accumulation 4, and train for 3 epochs. The LoRA rank is 32, the LoRA scaling factor is 64, and the LoRA dropout is 0.05.

GRPO For the GRPO baseline, we use the VERL framework and initialize the policy directly from the base model. We use group size 8 and rollout batch size 2. The learning rate is 1×10^{-6} , and we train for one epoch. We apply actor-side KL regularization with coefficient 0.001, but do not include

the KL term in the reward. The reward is based on the environment cumulative reward. Invalid actions receive a penalty of -0.1 , and premature submission receives a penalty of -0.2 .

MemRL For MemRL, we follow its non-parametric runtime memory reinforcement learning setup, where learning is performed through episodic memory updates rather than model weight updates. We use Qwen3.5-9B as the LLM backend and Qwen3-Embedding-0.6B as the embedding model. We retrieve at most 3 memories per query. The memory construction, retrieval, and update strategies are proceduralization, query, and adjustment, respectively. For Q-value updates, we use a success reward of $+1.0$ and a failure reward of 0.0 . Other MemRL reinforcement-learning hyperparameters are kept at the package defaults.

SkillZero For SkillZero, we use its skill-conditioned GRPO training setup implemented with the VERL framework. Skill conditioning is enabled during training. We initialize SkillZero with a MiniHack-specific skill set and use a curriculum schedule that starts with all skills, switches to core skills after step 100, and removes skills after step 200. The rollout group size is 4, the learning rate is 1×10^{-6} , and the batch size is 2. We train SkillZero for one GRPO epoch. Invalid actions receive a penalty of -0.1 .

ComplementaryRL ComplementaryRL is implemented with the ROLL framework under `MODE=complementaryrl`. It uses a fully asynchronous actor-memory pipeline with three non-shared modules: a Qwen3.5-9B actor, a Qwen3-4B-Thinking-2507 memory model, and a Qwen3-4B-Thinking-2507 memory actor, with Qwen3-Embedding-0.6B for retrieval. The actor is trained with GRPO, while the memory actor is trained with CISPO and REINFORCE-style advantage estimation. We use learning rate 1×10^{-6} , rollout batch size 128, group size 1, gradient accumulation 64, and 512 training steps. Actor rollouts use temperature 0.99, top- p 0.99, and at most 4096 new tokens per step. The memory system stores trajectory memories, retrieves the top-1 memory by embedding similarity, uses FIFO updates, and caps the memory pool at 20,000 items. KL regularization is disabled, and episodes are limited to 80 actions per trajectory.

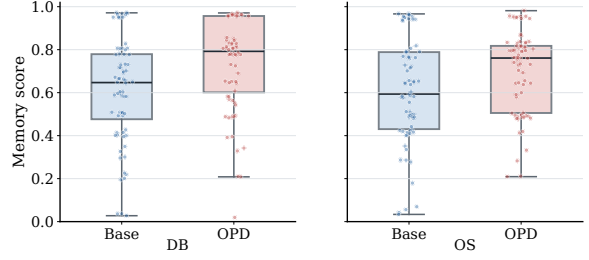


Figure 6: Additional results for memory selection on LifelongAgentBench. Left: DB; right: OS.

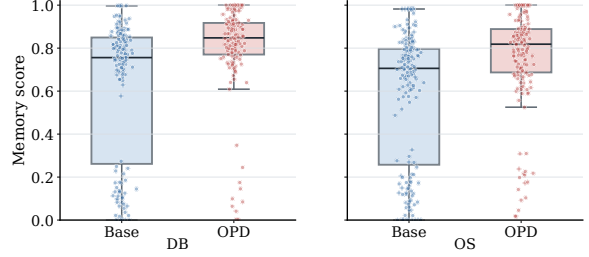


Figure 7: Additional results for memory writing on LifelongAgentBench. Left: DB; right: OS.

OPD-Evolver For OPD-Evolver, the memory pool contains four types of memories: skills, tips, tools, and trajectories. During teacher-side retrieval, we first retrieve 50 candidate memories and then select 20 memories for injection. For teacher memory filtering, we use a minimum score threshold of 0.01. For writer-side memory selection, we keep the top 30% of generated memories according to the memory score. We set the maintenance period to $Q = 30$ tasks. The evolver is optimized with the unified slow-loop objective described in Section 3.3.

The main experiments are conducted on a Linux server with 8 NVIDIA A800 (80G) GPUs.

E More Results

Figures 6 and 7 report additional results on LifelongAgentBench. These experiments complement the main results by evaluating the selection and writing distillation effects on the lifelong DB and OS environments. OPD-Evolver consistently shifts the memory-score distributions upward compared with the vanilla model, indicating that slow-loop training improves both which memories are injected and what memories are written for future reuse.

Impact of Selection Distillation on LifelongAgentBench. Figure 6 compares the calibrated memory-score distributions of memories selected by the vanilla model and by OPD-Evolver on Life-

longAgentBench DB and OS tasks. On both environments, OPD-Evolver yields higher-score selected memories than the base model. The score distribution shifts upward, and the low-score region is reduced, showing that the trained selector is less likely to inject broad, noisy, or weakly related memories. This supports the claim that selection distillation improves not only the amount of retrieved memory, but also the task relevance of the injected memory context.

Impact of Writing Distillation on LifelongAgentBench. Figure 7 evaluates whether the trained writer produces memories with higher future utility on LifelongAgentBench. Compared with the vanilla writer, OPD-Evolver produces memories whose calibrated scores are higher and more concentrated on both DB and OS tasks. The reduction of low-score memories suggests that the trained writer is less prone to storing generic or misleading advice, and instead writes compact memories that better capture the causal error patterns and task-specific constraints needed for future attempts. Together with the main results, these LifelongAgentBench experiments show that slow-loop training generalizes beyond the primary benchmark settings and improves both memory selection and memory writing in long-horizon agent environments.

F Case Study

Figures 8 to 10 provide qualitative examples of how slow-loop training improves the three components of the memory evolution pipeline: execution, memory selection, and memory writing. Across InterCode-SQL, LifelongAgentBench-OS, and MiniHack, the vanilla models often either over-generalize from previous experience or fail to localize the immediate cause of failure. In contrast, OPD-Evolver produces more task-specific behavior: it retrieves memories that match the current task, writes compact causal tips instead of generic advice, and executes actions that better satisfy environment-specific constraints.

Figure 8 focuses on the EXECUTOR. The examples show that the vanilla executor can violate task-level constraints, stop before the true success condition is reached, or omit required side effects. In InterCode-SQL, it issues multiple statements in one action and continues schema exploration after noisy observations. In MiniHack, it submits while merely adjacent to the goal. In LifelongAgentBench-OS, it completes group membership but misses the re-

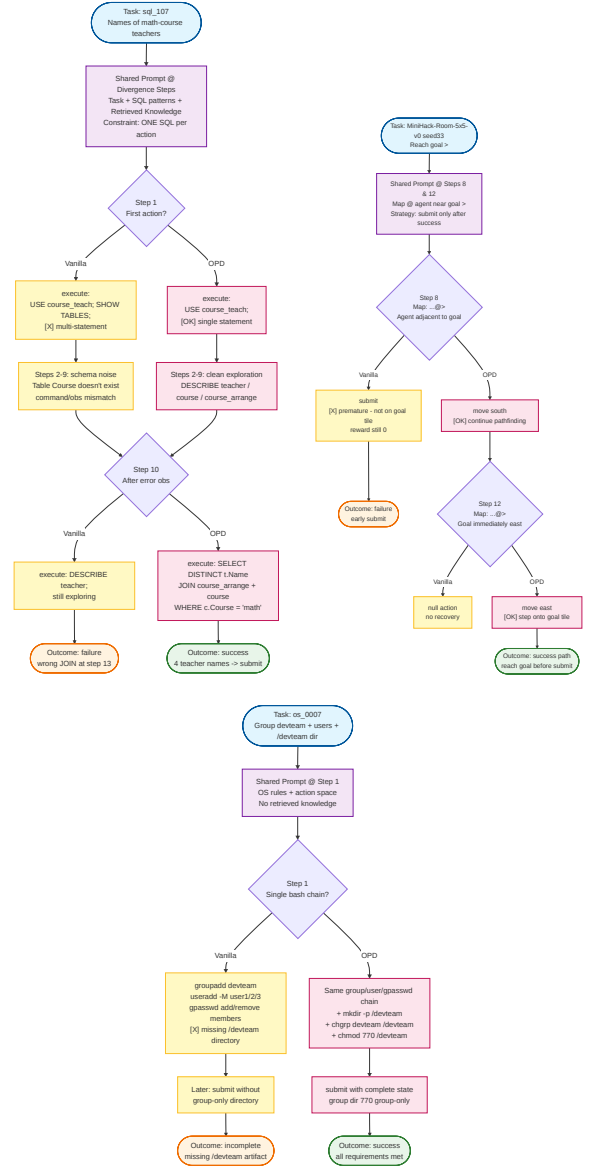


Figure 8: Case studies for the EXECUTOR. Top-left: InterCode-SQL; top-right: MiniHack-Room-5×5; bottom: LifelongAgentBench-OS.

quired group-owned directory. OPD-Evolver instead learns to respect the operational constraints of each environment: it separates SQL actions, continues navigation until the agent reaches the goal tile, and performs the missing directory creation and permission-setting operations.

Figure 9 examines the SELECTOR. The vanilla selector often injects memories that are superficially related but not causally useful for the current task, such as choosing a minimum-count SQL trajectory for a paragraph-counting query or a user/group setup workflow for a permission-editing OS task. In the MiniHack-Trap example, it even produces an empty injection after a JSON parse

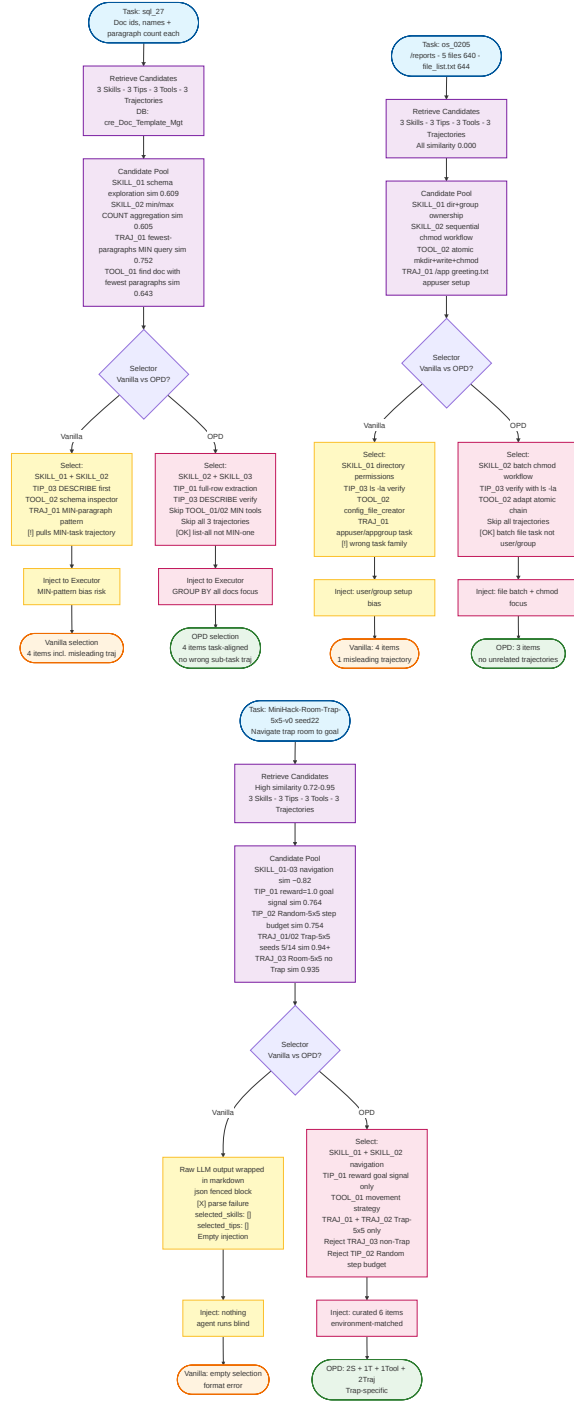


Figure 9: Case studies for the SELECTOR. Top-left: InterCode-SQL; top-right: LifelongAgentBench-OS; bottom: MiniHack-Trap.

failure. OPD-Evolver filters memory more aggressively and keeps only the skills, tips, tools, or trajectories that directly support the present problem. This indicates that slow-loop training improves not only whether memory is used, but also which memory is allowed to influence the next attempt.

Figure 10 analyzes the WRITER. The vanilla writer tends to convert failures into broad reusable

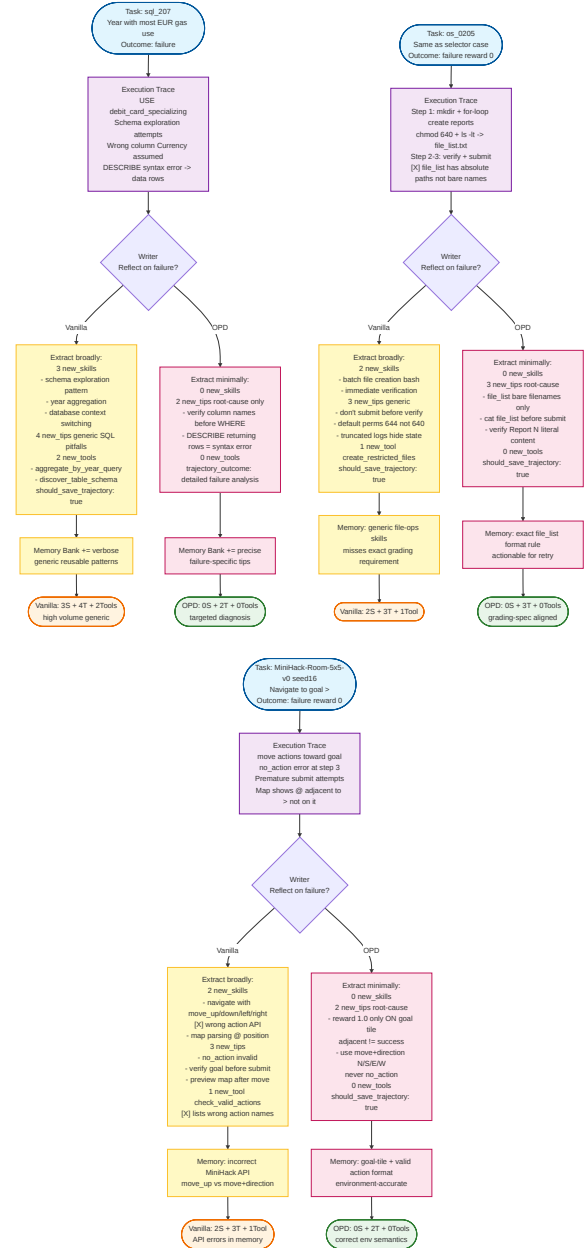


Figure 10: Case studies for the WRITER. Top-left: InterCode-SQL; top-right: LifelongAgentBench-OS; bottom: MiniHack-Room-5x5.

advice, and in some cases even records incorrect environment APIs. OPD-Evolver instead writes narrower future-facing memories that target the root cause of the failure. In InterCode-SQL, it records the need to verify column names and interpret malformed schema queries. In LifelongAgentBench-OS, it records the grading-specific requirement to output bare filenames and verify the generated file list before submission. In MiniHack, it records the correct action interface and the fact that reward is obtained only by stepping onto the goal tile.

G Information About Use Of AI Assistants

AI assistance was used only for writing polishing and auxiliary coding support in this work.